

VLSI Implementation of Fully Parallel and CSD FIR Filter Architecture

Prabhavathy M
SENSE
VIT

Chennai, India
prabhavathy.m2020@vitstudent.ac.in

Dr.Sakthivel S M
SENSE
VIT

Chennai, India
sakthivel.sm@vit.ac.in

Abstract—This paper proposes the VLSI implementation of Fully parallel and CSD FIR filter architecture. In this technique, the area and power optimization are achieved by the incorporation of Vedic multiplier and Kogge stone adder instead of traditional multiplier and adder. Furthermore, a valid comparison of aforesaid architecture for 8-tap and 16-tap FIR filters is made in both linear and tree topology. Further, the proposed filter architecture is also analyzed in the presence of a pipeline mechanism. The VLSI implementation of the proposed FIR filter using Verilog HDL is carried out both in FPGA (Kintex -7 xc7k70tbbv676-1 family) and ASIC (180 nm technology) using Xilinx Vivado and Cadence Genus respectively. The FPGA implemented architecture consumes 1513 LUTs and 184 Slices for 8- tap and 16-tap. Similarly, the ASIC implementation occupies 93668 μm^2 area and 15.254 mW power.

Keywords— *Finite impulse response filter, Canonical Signed Digit, Fully Parallel, Quantized co-efficient, Vedic multiplier & Kogge stone adder.*

I. INTRODUCTION

In general FIR filters are majorly preferred over IIR filters because of their stability and linear phase response [1-2]. Also, in most of the signal processing applications like speech and biomedical FIR filters are used. Further, the FIR filters are intended for ECG signal denoising [3], EEG filtering [4], and Seismic signal processing [5]. Generally, FIR filtering operation involves addition and multiplication which is realized using adder, delay, and multiplier components during VLSI implementation [6]. Numerous FIR filter structures are proposed over the past decade in the literature are Energy Efficient FIR filter [7], a parallel-based FIR filter [8], and an optimized Multiplier based FIR filter [9]. In all the aforementioned architectures the filter structures focus mainly on the aspect of signal filtering efficiency only but not on the hardware performance.

To overcome the above limitation an area and power-efficient FIR filter design are proposed using the Vedic multiplier and Kogge stone adder. The reason for choosing the Vedic multiplier is due to its fast operation and low power consumption [10] similarly Kogge stone adder is incorporated to improve the timing as well as power [11]. Additionally, in this research paper, a valid performance comparison of Fully parallel [12] and CSD architecture with and without pipeline is carried out and analyzed.

This research paper is organized as follows, In section-1, the general overview of FIR filter architecture and the summary of reported FIR filter structures in the literature are discussed. Section-2 elaborates on the problem

statement related to this research work. Section-3 discusses the Fully parallel and CSD proposed FIR structure using Vedic multiplier and Kogge stone adder. In section-4 the experimental results and discussion of proposed filter structures are given in detail.

A. RELATED WORK

Several FIR filter architectures have been proposed using Vedic multipliers and several adders to improve the performance in terms of area, power, and delay. For instance, samyuktha et.al designed an efficient FIR filter using a Vedic multiplier and a hybrid ripple carry adder to reduce the number of LUTs [13]. Consecutively, low power and area efficient FIR filter structure has been proposed by sumalatha et.al using Vedic mathematics and Carry look-ahead adder [1]. Following sumalatha et.al an optimized FIR filter using reversible logic gate is implemented by Janaki Ram et.al [14] and Ravi et.al [15]. Though the aforesaid architectures consume less power and area, still there are chances to improve the above parameters. Henceforth in the proposed work, area and power optimization are achieved using CSD-Tree architecture.

II. PROBLEM STATEMENT

In this section, the difficulties and the solution to the proposed FIR filter are discussed in detail. In the basic operations of FIR filter addition and multiplication plays a significant role. Generally, the multiplication process involves partial product generation and its reduction. Hence the multiplication operation increases the hardware complexity in terms of area and power. To reduce the hardware complexity in terms of area and power an efficient CSD-Tree FIR filter design using Vedic multiplier and Kogge stone adder is incorporated in the proposed work. Thus the implemented CSD FIR filter outperforms well in terms of area and power when compared with the conventional FIR structure.

III. PROPOSED FIR FILTER ARCHITECTURE

This section explains the proposed filter structure with and without a pipeline. Additionally in the proposed FIR filter realization tree and linear structure topology are adopted.

A. Proposed Filtering Unit And Its Operation

Further in this section a detailed filtering unit, explanation of CSD architecture, and incorporated Vedic multiplier and kogg stone adder structures are discussed.

1) *Fully parallel*: The basic architecture of a fully parallel structure involves a multiplier and an accumulator unit and a delay element (Z^{-1}) as represented in figure 1 for each tap. Here $X(n)$, $Y(n)$, and $(C_0, C_1, C_2, \dots, C_N)$ are the input, output and filter coefficients respectively [16-17]. Thus the fully parallel structure for the N-tap filter consists of N coefficients, N multiplier & N-1 adder elements similar to the direct form structure.

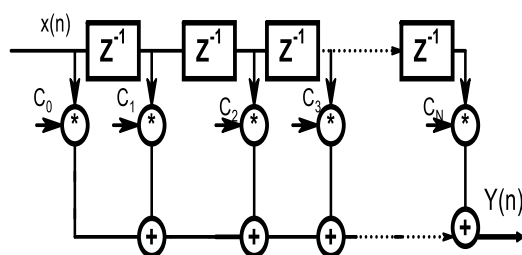


Fig 1: Basic block of FIR Filter

The basic architecture of a pipelined fully parallel architecture [18-19] is shown in Fig 2, Here an extra block (P) is added as a pipeline unit before every delay element (input pipeline) and after each multiplier unit (output pipeline).

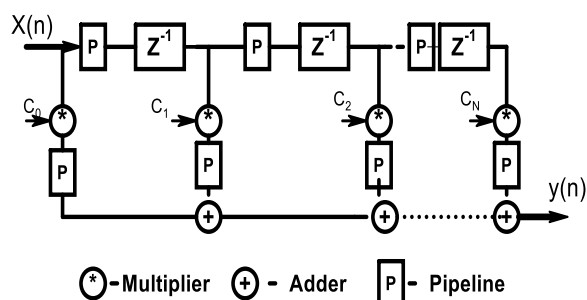


Fig 2: Pipelined Fully parallel linear FIR structure

Here adder operation is performed linearly as the structure is in a linear topology. Next, a tree-based topology is adopted for the proposed FIR structures as shown in Fig 3. Thus in the tree-based structure, the last coefficient is getting added to the next coefficient, and it is iteratively repeated for all coefficients. If this tree-based structure has opted for odd tap filters the leftover coefficient is added to the sum. Similarly, there will be no leftover coefficient for even tap filters.

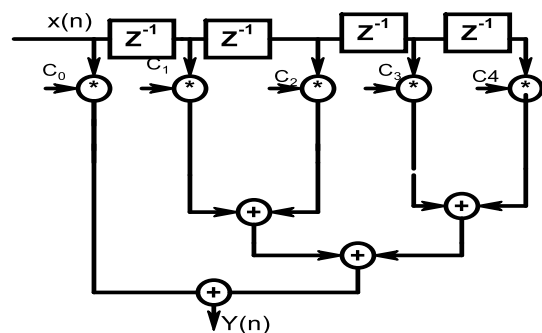


Fig 3: Tree-based Fully parallel FIR structure

2) *CSD*: It is a canonical signed digit representation of the filter coefficients. In this, the binary values of a coefficient are represented with $\{0, 1, -1\}$. Because of $\{0, 1, -1\}$ CSD-based filter realization [18-19] involves minimal non-zero terms which primarily leads to less complex hardware implementation using shifting operations compared to its binary representation [20-21]. The CSD structure of FIR filter using Tree and linear topology are given in Fig. 4 & 5 respectively.

For simplicity, a 5-tap filter is considered for an explanation. Normally in 5 tap FIR there exist five coefficient namely $h(0), h(1), h(2), h(3)$ and $h(4)$. During the realization of the filter here the coefficients $h(0)$ and $h(4)$ are selected as the same using a symmetric property. Here the CSD method of representing a filter coefficient is explained below with an example. For the filter coefficient 2287, the binary representation and the corresponding CSD representation are given below.

For coefficients, $h(0)=h(4)$

Binary representation: 16'b0000100011101111

CSD representation: $2^{11} + 2^8 - 2^5 + 2^4 - 2^0$

i.e 0000100100-11000-1

similarly, for all the other coefficients same CSD representation is adopted.

For coefficients, $h(1)=h(3)=h(2)$

Binary representation: 16'b0110111000100001

It's CSD: $2^{15} - 2^{13} + 2^{12} - 2^9 + 2^5 + 2^0$

i.e 10-1100-1000100001

For the filter structure using CSD [22] representation of considered coefficient (2287), there will be five non-zero terms with three adder and two subtractor networks. Likewise, the corresponding number of adder and subtractor units are included using the respective coefficient in the filter structure. The 5 tap CSD-Tree FIR filter structure for the filter coefficients 2287 with Tree topology and linear topology is given in Fig 4.&5 respectively. On comparing the above structure with fully parallel FIR filter architecture, it varies only in the multiplier block using shift operation performed by adder or subtractor network.

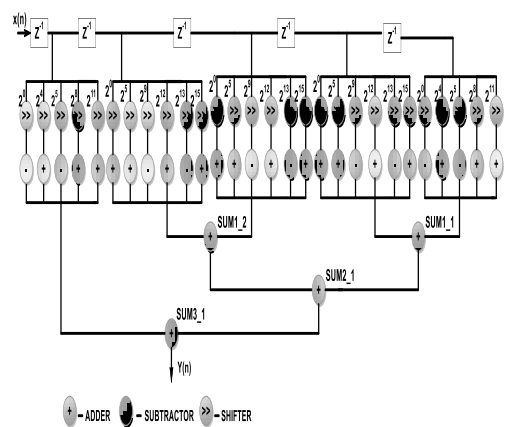


Fig 4: 5-tap CSD-Tree FIR filter structure

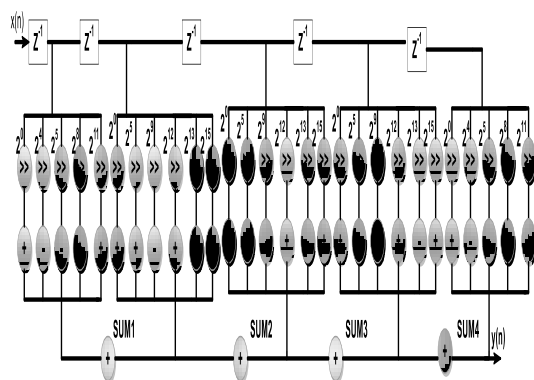


Fig 5: 5-tap FIR Filter CSD-Linear FIR filter structure

B. Proposed Vedic Multiplier

From the basics of FIR filter, it is identified that multiplication operation consumes more time which in turn increases the area and power of the filter architecture. Therefore to address this in the proposed work Urdhva Tiryagbhyam-based Vedic multiplier [1,3] is incorporated. Due to the incorporation of Vedic multiplier-based FIR filter realization the proposed filter structure consumes low power and delay. The main reason for low power and area in Vedic multiplier structure is due to its partial product calculation using Urdhva Tiryagbhyam sutra (i.e. it consumes comparatively lesser hardware components than the traditional multiplier).

$$\begin{array}{rcl} \begin{array}{cc} a1 & a0 \\ b1 & b0 \end{array} & \begin{array}{c} | \\ | \end{array} & - S0 = a0b0 \\ \begin{array}{cc} a1 & a0 \\ b1 & b0 \end{array} & \begin{array}{c} \diagdown \\ \diagup \end{array} & - C1S1 = a0b0 + a0b1 \\ \begin{array}{cc} a1 & a0 \\ b1 & b0 \end{array} & \begin{array}{c} | \\ | \end{array} & - C2S2 = a1b1 + c1 \end{array}$$

Fig 6: 2 x 2 Vedic Multiplication

For explanation here, a 2 x 2 Vedic multiplier [23] is considered in which a0, a1, and b0, b1 is the multiplier and multiplicand, respectively. The entire 2 x 2 multiplication is carried out in two steps Vertical (a0b0 and a1b1) and crosswise (a1b0 and a0b1) multiplication [5,9]. Finally, the

result of vertical and crosswise is added. The same kind of operation is extended for the 4 x 4 Vedic multiplier using two by two multiplier. The same Vedic multiplier operation has been pictorially represented in Fig 6. and 7 respectively.

$$\begin{array}{rcl} \begin{array}{cc} a3 & a2 \\ b3 & b2 \end{array} \begin{array}{c} \begin{array}{cc} a1 & a0 \\ b1 & b0 \end{array} \\ \begin{array}{cc} a1 & a0 \\ b1 & b0 \end{array} \end{array} & - & (a1a0+b1b0) = S0 \\ \begin{array}{cc} a3 & a2 \\ b3 & b2 \end{array} \begin{array}{c} \begin{array}{cc} a1 & a0 \\ b1 & b0 \end{array} \\ \begin{array}{cc} a2 & a1 \end{array} \end{array} & - & (a3a2+b0) * c1 = S1 * C1 \\ \begin{array}{cc} a3 & a2 \\ b3 & b2 \end{array} \begin{array}{c} \begin{array}{cc} a1 & a0 \\ b1 & b0 \end{array} \\ \begin{array}{cc} a2 & a1 \end{array} \end{array} & - & (a1a0+b3b2) * c2 = S2 * C2 \\ \begin{array}{cc} a3 & a2 \\ b3 & b2 \end{array} \begin{array}{c} \begin{array}{cc} a1 & a0 \\ b1 & b0 \end{array} \\ \begin{array}{cc} a2 & a1 \end{array} \end{array} & - & (a3a2+b3b2) * c3 = S3 * C3 \end{array}$$

Fig 7: 4 x 4 Vedic Multiplication using Urdhva Tiryagbhyam sutra

Based on the aforementioned 4 x 4 Vedic multiplication operation, the VLSI structure for the multiplier has been constructed using four 2 x 2 Vedic multipliers and three adder blocks. The corresponding structure for the 4 x 4 multiplier is given in Fig 8 using kogg stone adder instead of the traditional adder.

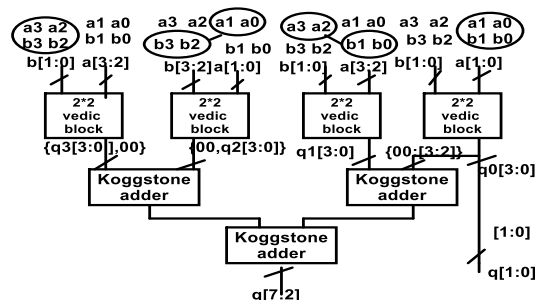


Fig 8: Block diagram of Vedic 4x 4 Multiplier

C. PROPOSED KOGGSTONE ADDER

Koggstone adder is a parallel prefix adder. These adders are obtained from the carry-look-ahead adder and are used to speed up the binary addition [24-25]. The kogg stone adder structure consists of Black cells, Grey cells, and buffers as represented in Fig 9. In this adder carry generation and carry propagation is carried out in Black and Grey cells unit respectively. Usually, in kogg stone adder, Black cells are followed by Grey cells. Generally in kogg stone adder, the addition operation is carried out in three stages as stated below [23-24]:

- Pre-processing stage
- Carry generation stage
- Post-processing

Pre-possessing stage: In this stage, the inputs Ai and Bi are used to produce the generate (Gi) and propagate (Pi) values which are represented in equations 1 & 2.

$$Pi = Ai \wedge Bi \quad (1)$$

$$Gi = Ai \& Bi \quad (2)$$

Carry generation stage (Black cell): The second stage involves the use of two consecutive generate & propagate (i.e GiPi, GjPj) as inputs to produce the output Carry Generate (CG0) and Carry propagate (CP0) are periods when they are part of a sentence, as in:

$$CP0 = Pi \& Pj \quad (3)$$

$$CG0 = (Pi \& Gj) | Gi \quad (4)$$

Post-processing stage (Grey cell):

In this final stage, the sum bits and carry bits are generated using equations 5 & 6. Normally this grey cell stage is added to improve the speed of the adder by calculating the carry ahead of the sum output.

This stage will generate two signals :

1. Computation of sum bits Si
2. Computation of carrying bits Ci

$$Si = Pi \wedge Ci_{-1} \quad (5)$$

$$Ci - 1 = (Pi \& C in) | Gi \quad (6)$$

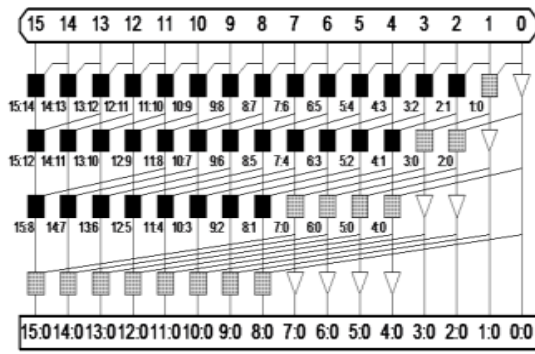


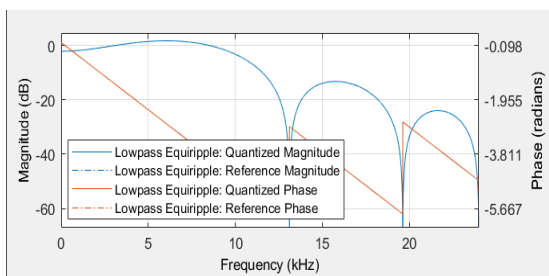
Fig 9: Structure of kogg stone adder

IV. FILTER DESIGN

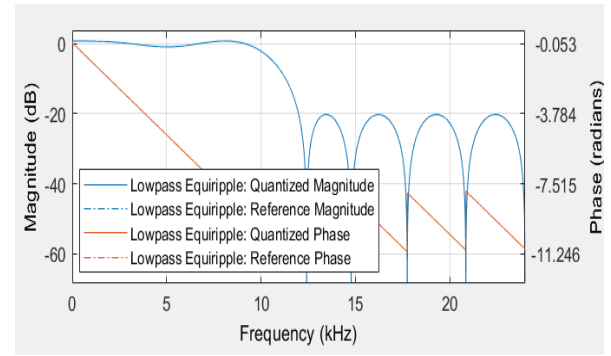
For identifying the FIR filter coefficient the filter designer tool(FDA) in Matlab is used in this work. Here for 8 tap and 16 tap filters, the filter coefficients are determined using the FDA tool for the below specification :

- Sampling frequency- 48 kHz
- Stop band - 12 kHz
- Cut-off frequency - 9600 Hz.

For the above specification the corresponding magnitude and phase response of the 8-tap and 16-tap FIR filter is shown in Fig 10(a) and 10(b). The magnitude response of the 8-tap filter has three lobes whereas 16-tap filters consist of 5 lobes.



(a)



(b)

Fig 10: Magnitude and phase response of (a) 8-tap (b) 16-tap LPF

The respective filter coefficients [18] for the 8-tap and 16-tap FIR filter is given below.

The following are the 8-tap coefficients :

$$h(0)=h(7)= 1101110110111011$$

$$h(1)=h(6)= 1110101010001110$$

$$h(2)=h(5)= 0011001111011011$$

$$h(3)=h(4)= 0110100000001000$$

The Filter coefficients for the 16-tap Filter obtained are :

$$h(0)= h(15)=1111001110011010$$

$$h(1)=h(14)= 0000100000011010$$

$$h(2)=h(13)= 0001000011111001$$

$$h(3)=h(12)= 0000010000100000$$

$$h(4)=h(11)= 1110110010000000$$

$$h(5)=h(10)= 1111010101001110$$

$$h(6)=h(9)= 0010111100001011$$

$$h(7)=h(8)= 0110101011001101$$

V. EXPERIMENTAL RESULTS AND DISCUSSIONS

This section discusses the results of 8-tap and 16-tap FIR filter VLSI implementation for Fully parallel and CSD Architecture. During the VLSI implementation initially, the filter is modeled using Verilog HDL code and simulated using Xilinx Vivado(FPGA)& Cadance Genus tool(ASIC). For this functional verification, ECG data values are considered as input.

Then using the functional verified RTL code of FIR filter the Fully parallel and CSD Architecture is implemented in both FPGA(Kintex -7 xc7k70tfbv676-1 family) and ASIC(180 nm). Also the 8 tap & 16 tap FIR filter structures are implemented with and without pipeline, then the respective VLSI implemented statistics are given in Tables 1 & 2 respectively.

It is inferred from Table 1, that the FIR filter without a pipeline architecture in CSD realization consumes less area & power for both FPGA & ASIC implementation[27]. For FPGA implementation using CSD structure leads to an 11% decrease in LUTs. Similarly for the ASIC implementation around 40% reduction in power with a 30% reduction in area.

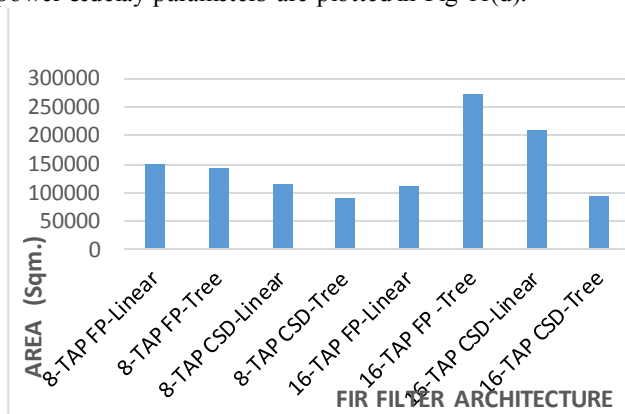
TABLE I. VLSI STATISTICS OF 8-TAP & 16-TAP FIR FILTER FOR FULLY PARALLEL & CSD STRUCTURES

Taps	FIR Filter structures	ASIC			FPGA	
		Area (μm^2)	Power (nW)	Delay (pS)	LUT	Slice Registers
8	Fully Parallel-Linear	148982	23027820	12790	1879	160
	Fully Parallel-Tree	144741	20079578	9881	1830	160
	CSD-Linear	114996	20061936	9909	1671	160
	CSD-Tree	91476	13352568	9909	1654	160
16	Fully Parallel-Linear	113533	25830624	9274	3472	288
	Fully Parallel-Tree	273293	41191916	9833	3372	288
	CSD-Linear	210966	52422710	17104	3105	288
	CSD-Tree	93668	15254216	9611	1513	184

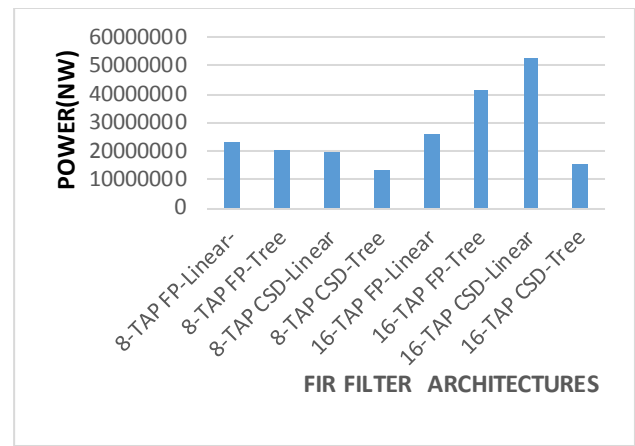
TABLE II. VLSI STATISTICS OF PIPELINED FIR FILTER FOR FULLY PARALLEL & CSD FIR STRUCTURES

Taps	FIR Filter structures	ASIC			FPGA	
		Area (μm^2)	Power (nW)	Delay (pS)	LUT	Slice Registers
8	Fully Parallel-Tree	154441	17887582	9881	1930	185
	CSD-Tree	91476	13352568	9835	1654	160
16	Fully Parallel-Tree	315346	45497963	983	3707	317
	CSD-Tree	93668	15254016	9611	1513	184

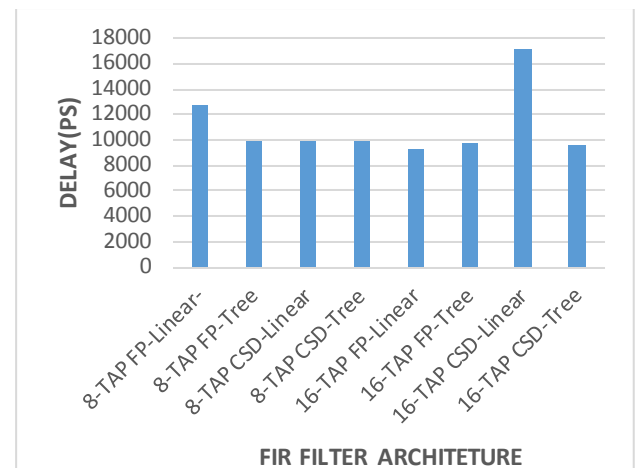
Similarly, for the pipelined structure a valid comparison is carried out for both 8-tap & 16-tap the respective VLSI statistics of FPGA & ASIC implementation are given in Table 2. From the VLSI implemented results, it is identified that there is a 50% reduction in LUTs for FPGA implementation, 41% area reduction, and 25% power reduction for ASIC implementation. Further from the tabulated values of ASIC & FPGA implementation, it is identified that delay remains the same for both pipelined and non-pipelined CSD structures. Further for the non-pipelined structure, the area, power and delay analysis for both fully parallel and CSD structures are plotted as a bar graph in Fig. 11(a), 11(b), and 11(c) respectively. Similarly, for the pipelined structure the ASIC implemented results of area, power & delay parameters are plotted in Fig 11(d).



(a)

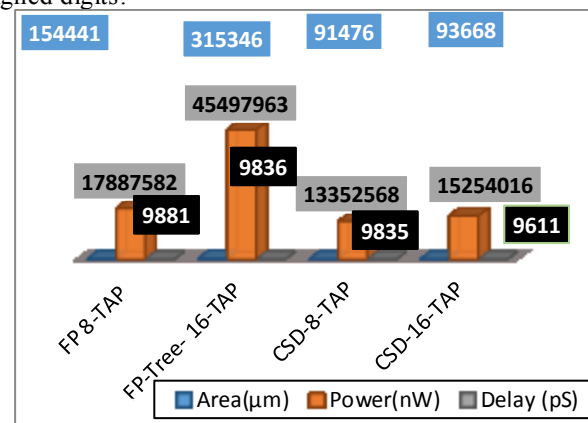


(b)

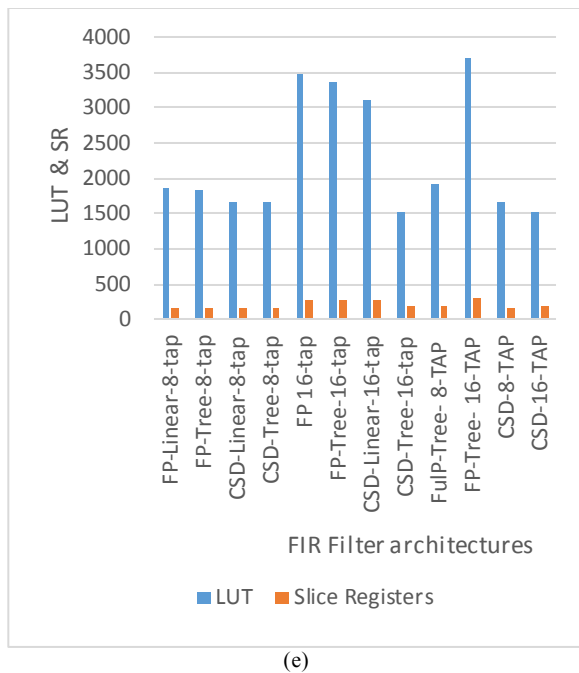


(c)

Then the FPGA implementation of both CSD and fully parallel structure is plotted in Fig 11(e). In Fig 11, FP represents Fully parallel, and CSD represents Canonical Signed digits.



(d)



(e)

Fig11: Performance comparison of proposed FIR architectures a)Area comparison Without Pipeline b) Power comparison Without Pipeline c) Delay comparison Without Pipeline d) Area, Power and Delay Comparison With Pipeline (e)Comparison of LUT and Slice registers

TABLE III. PERFORMANCE COMPARISON OF PROPOSED FIR STRUCTURE WITH OTHER REPORTED ARCHITECTURE IN THE LITERATURE

FIR Filter N-tap	FIR Filter architecture	Power (mW)
8	The normal architecture of the Reversible VD-CLA Filter[14]	247.94
	The normal architecture of the Reversible VD-CSLA Filter[14]	247.584
	Proposed architecture	13.352
16	The normal architecture of the Reversible VD-CLA Filter [14]	247.884
	The normal architecture of the Reversible VD-CSLA Filter [14]	247.384
	Proposed architecture	15.254

Then a valid performance comparison of the proposed technique is carried out with other reported FIR filter structures in the literature using the power parameter. The respective values of power for each of the methods are given in Table 3 for the non-pipelined mode. The tabulated values of power clearly show that the proposed architecture is having a low power of 13.35mW & 15.25 mW for 8-tap and 16-tap pipelined FIR filters respectively.

For simulating and verifying the functionality the 3000 sample of ECG data is considered as input. First, the filter functionality is verified using MATLAB code simulation and then the VLSI simulation & implementation is performed. The simulated output response of CSD tree FIR filter architecture is given in Fig 12 for the considered ECG signal input. Likewise, the RTL schematic of the 16-tap CSD tree FIR filter architecture is given in Fig 13.

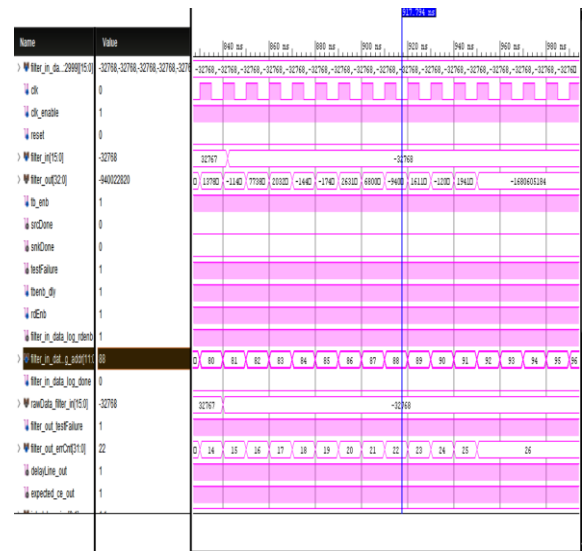


Fig 12: The output waveform of 16-tap CSD -Tree FIR filter architecture

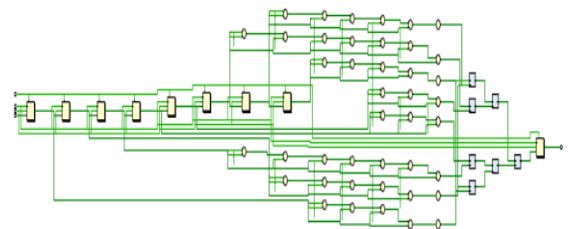


Fig 13:RTL Schematic of 16-tap CSD-Tree FIR filter architecture

VI. CONCLUSION

In this research, an 8-tap and 16-tap FIR filter VLSI implementation is carried out for CSD tree architecture and Fully parallel architecture. Further, the area and power reduction in the proposed technique is achieved by the incorporation of the Vedic multiplier and Kogge stone adder. Then a performance comparison of Fully parallel and CSD architecture both in linear and tree-based topology is carried out for each of the structures. Furthermore, the filter structures are analyzed with and without pipeline mechanisms to study their performance. The VLSI implemented CSD architecture consumes comparatively less power and area when compared with Fully parallel architecture. The FPGA implementation using Kintex -7 xc7k70tbfv676-1 family consumes 1513 LUTs and 184 Slices for 8-tap & 16-tap filters. Likewise, the ASIC (180 nm technology) Cadence Genus occupies a 93668 μm^2 area and 15.254 mW power.

VII. LIMITATIONS AND FUTURE SCOPE

A. Limitations:

Even though the proposed CSD-based FIR filter structures consume less area and power, still there are possibilities for delayed improvement in the proposed architecture. Similarly

for the linear structure though the delay is low still there exist possibilities of the area & power optimization.

B. Future Scope:

Further, the filter performance has been improved by incorporating hybrid optimization of co-efficient prediction. Also, we can improve the timing (delay) of the filter by retiming techniques. Then in the future, it can be explored for ECG filtering applications, incorporating a Genetic algorithm, Reversible logic gates, and the design of a 2D-based filter. Also, the EEG application can be extended.

VIII. ACKNOWLEDGEMENT

With great pleasure and genuine gratitude, we would like to thank the Vellore Institute of Technology Chennai campus for providing us with the infrastructure and numerous other resources we needed for this project.

REFERENCES

- [1]. M. Sumalatha, P.V. Naganjaneyulu, K. Satya Prasad, Low power and low area VLSI implementation of Vedic design FIR filter for ECG signal de-noising, *Microprocessors and Microsystems*, Volume 71, 2019, 102883, ISSN 0141-9331
- [2]. Subhabrata Roy, Abhijit Chandra A Survey of FIR Filter Design Techniques: Low-complexity, Narrow Transition-band, and Variable Bandwidth, *Integration*, Volume 77, 2021, Pages 193-204, ISSN 0167-9260.
- [3]. Partial product addition in Vedic design-ripple carry adder design fir filter architecture for electro cardiogram (ECG) signal denoising application, *Microprocessors and Microsystems*, Volume 76, 2020, 103113, ISSN 0141-9331
- [4]. A new approach for the design of digital frequency selective FIR filters using an FPA-based algorithm, *Expert Systems with Applications*, Volume 106, 2018, Pages 92-106, ISSN 0957-4174,
- [5]. Area-Delay-Power Efficient VLSI Architecture of FIR Filter for Processing Seismic Signal, *IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—II: EXPRESS BRIEFS*, VOL. 68, NO. 11, NOVEMBER 2021
- [6]. Seshadri, R. & Ramakrishnan, S., Fpga implementation of fast digital fir and iir filters, *Concurrency and Computation: Practice and Experience* 33(3), e5246, 2021.
- [7]. Ranjith, C. & Rani, S. J. V., A fast on-chip adaptive genetic algorithm processor for evolutionary fir filter implementation using hardware-software co-design, *Journal of Circuits, Systems and Computers* 29(01), 2050014, 2020
- [8]. Sang Yoon Park, and Pramod Kumar Meher, "Efficient FPGA and ASIC realizations of DA-Based reconfigurable FIR digital filter", *IEEE Transactions on Circuits and Systems-II: Express Briefs*.
- [9]. Gaur, N., Kapur, S. & Mehra, A., Application of vedic multiplier: Design of a fir filter, in *2020 4th International Conference on Electronics, Communication and Aerospace Technology (ICECA)*, IEEE, pp. 234-237, 2020.
- [10]. Yancang Chen, Minlei Zhang Pei Wei1, Sai Sui1, Yaxin Zhao1 and Lunguo Xie2, Implementation of a Parallel Prefix Adder Based on Kogge-Stone Tree International Conference on Communications, Information Management and Network Security (CIMNS2016)
- [11]. S. N. S. Vishnu, A. Gandluru and R. S. R., "32-Bit RISC Processor Using Vedic Multiplier," *2022 3rd International Conference for Emerging Technology (INCEET)*, 2022, pp. 1-5.
- [12]. V. Sudhakar, N. S. Murthy and L. Anjaneyulu, "Fully parallel and fully serial architecture for realization of high speed FIR filters with FPGA's," *2012 International Conference on Devices, Circuits and Systems (ICDCS)*, 2012, pp. 499-501, doi: 10.1109/ICDCSyst.2012.6188766.
- [13]. S. Samyuktha, D.L. Chaitanya, VLSI design of efficient FIR filters using Vedic Mathematics and Ripple Carry Adder, *Materials Today: Proceedings*, Volume 33, Part 7, 2020, Pages 4828-4832, ISSN 2214-7853
- [14]. E. Janaki Ram, S. Swetha, Implementation of optimized FIR filter using reversible logic gates, *CVR Journal of science and technology* Vol 19 No 1 (2020): CVR Journal of Science and Technology
- [15]. Ravi JN*, Vijay Prakash AM and Madan S, Design and Implementation of Vedic Multipliers Using Reversible Logic Gates, *J Electr Electron Syst* 2018, 7:2 DOI: 10.4172/2332-0796.1000255
- [16]. Patali, P. & Kassim, S. T., High throughput fir filter architectures using retiming and modified csla based adders, *IET Circuits, Devices & Systems* 13(7), 1007-1017,
- [17]. Chandra A, Chattopadhyay S, Design of hardware efficient FIR filter: a review of the state of the art approaches. *Int J* 19:212-226, 2015
- [18]. Mathworks, HDL Serial architectures for Filters, <https://in.mathworks.com/help/hdlfilter/hdl-serial-architectures-forfir-filters.html>
- [19]. Pramod Patali*, Shahana Thottathikkulam Kassim, High throughput and energy efficient linear phase FIR filter architectures, *ScienceDirect, Microprocessors and Microsystems* 87 (2021) 104367
- [20]. K. V. Gowreesrinivas and P. Samundiswaray, "Implementation and Analysis of Single Precision Floating Point Multiplication Using Vedic and Canonic Signed Digit Algorithm," *2018 9th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, 2018, pp. 1-4, doi: 10.1109/ICCCNT.2018.8494184.
- [21]. Gustavo A. Ruiz, Mercedes Granda, Efficient canonic signed digit recoding, *Microelectronics Journal*, Volume 42, Issue 9, 2011, Pages 1090-1097, ISSN 0026-2692
- [22]. Srivastava, A. K. & Raj, K., An efficient fir filter based on hardware sharing architecture using csd coefficient grouping for wireless application, *Wireless Personal Communications* pp. 1-16, 2021.
- [23]. Jayashree S, Shantha preetha, Sakthivel S M, FPGA implementation of optimized FIR filter for ECG denoising using peres reversible logic gate, *Journal of Theoretical and Applied Information Technology*, 15th August 2022, Vol 100, No. 15
- [24]. Lee Mei Xiang, Muhammad Mun'im Ahmad Zabidi, Ainy Haziyah Awab, Ab Al-Hadi Ab Rahman, VLSI Implementation of a Fast Kogge-Stone Parallel-Prefix Adder, *IOP Conf. Series: Journal of Physics: Conf. Series* 1049 (2018) 012077
- [25]. Yancang Chen, Minlei Zhang Pei Wei1, Sai Sui1, Yaxin Zhao1 and Lunguo Xie2, Implementation of a Parallel Prefix Adder Based on Kogge-Stone Tree International Conference on Communications, Information Management and Network Security (CIMNS2016)
- [26]. Karuppusamy, P. "Design and Analysis of Low-power. High-Speed Baugh Wooley Multiplier." *Journal of Electronics* 1, no. 02 (2019): 60-70.
- [27]. Bansal, Malti, Harmandeep Singh, and Gaurav Sharma. "A Taxonomical Review of Multiplexer Designs for Electronic Circuits & Devices." *Journal of Electronics* 3, no. 02 (2021): 77-88.